

HEAR YE, HEAR YE



TECHORAMA

◆ MEDIEVAL EDITION ◆



TECHORAMA

2026 MEDIEVAL EDITION

What the Scrolls Didn't Say

A Real-World Microsoft Agent Framework Quest

A Few Heroes Walk Into a Tavern

Frederiek Vandepitte



in/fvandepitte



fvandepitte



Wouter Bauchart

in/wouterbau



wouterbau



The Request

Our most **seasoned knights** are no longer forging new tools but instead **inspecting the work of apprentices**.

Endless **reviews**. Endless **judgement**.

Could a construct **assist** in judging the quality of this work, and restore balance to the guild?



Our Response

*Yes, we can help restore balance.
But we would not build a simple tool.*

*We would build an **intelligent** construct
One capable of assisting in the **judgement** itself...*

*One that promises **great power!***



The Real Request

Our customers has

*An **Internal Developer Platform***

*Hosts **Tools** and **Processes***

*Supports existing **Azure DevOps***

***International** and **Multi-Timezone** Teams*



The Real Request

Was searching help to

***Optimize work** for Senior Developers*

***Speed up development** feedback cycle*

***Shift Left** for all Developers*

*Ensure **consistency** in work*



The Real Request

A Proof of Concept was requested to

***Automate feedback** on DevOps YAML Pipelines*

Ensure development standards

Give direct feedback

Reduce senior review load

Help developers move faster



Our Real Response

Who has already added AI to their project?

Got asked to put AI in your application?



Our Real Response

*A perfect use case for **AI Agents***

Recognize patterns

Research and compare best practices

Generate a human readable report

Generate a machine interpretable report



Our Real Response

*The **AI Agents** belong in the **Developer Platform***

Provided through a (.NET) webhost

To integrate in

Automation (OpenAI)

Developer Environments (MCP)

Tooling (AG-UI)

***Central Management** of agents*

*Providing usage **Observability** and **Control***



Our Real Response

*The **perfect solution** could be made with a framework in **preview**...*

Who has worked with system in preview as well?

But why do we take this risk?

Competitive Edge

Maintainer promises

Fits the use case

Speed of AI innovation

Human curiosity



Infusing AI and Agents...

Who has worked with Semantic Kernel?

Who already created an AI Agent with it?

Who has worked with AutoGen?



Semantic Kernel Agent

```
#pragma warning disable SKEXP0070
var builder = Kernel.CreateBuilder();
builder.AddOllamaChatCompletion(model, new Uri(uri));
var kernel = builder.Build();

var agent = new ChatCompletionAgent()
{
    Name = "Story Agent",
    Kernel = kernel,
    Instructions = "You are a helpful assistant that creates funny creative stories."
};

await foreach (ChatMessageContent response in agent.InvokeAsync(
    "Describe a funny story of 3 lines about a dog barking at a cat"))
{
    Console.WriteLine($"{response.Content}");
}
```


Microsoft.Extensions.AI

Who uses Microsoft.Extensions.AI?

NuGet Package

Unified generative AI components

Abstraction over different AI Model Services

Been around since May 2025

Became a base component



Microsoft.Extensions.AI

```
using Microsoft.Extensions.AI;  
using OpenAI.Chat;  
...  
IChatClient chatClient = new ChatClient(model, apiKey).AsIChatClient();  
ChatResponse response = await chatClient.GetResponseAsync("Hello World!");  
Console.WriteLine(response.Text);
```


Microsoft.Extensions.AI

```
using Microsoft.Extensions.AI;  
using OllamaSharp;  
...  
IChatClient chatClient = new OllamaApiClient(uri, model);  
ChatResponse response = await chatClient.GetResponseAsync("Hello World!");  
Console.WriteLine(response.Text);
```


Microsoft Agent Framework

*Created for **.NET** and **Python***

*Based on **Semantic Kernel** and **AutoGen***

*Uses **Microsoft.Extensions.AI***

Reuse existing abstractions and functionalities

Turn current AI clients into AI Agents

*Go from **Generative AI** to **AI Cooperation***

***Agent definitions** to perform actions*

*Orchestrate **Workflows** of agent interactions*

Perfect to create focused AI solutions for the request!



Microsoft Agent Framework

```
var chatClient = new OllamaApiClient(uri, model);  
var agent = chatClient.AsAIAgent(  
    instructions: "You are a helpful assistant that creates funny creative stories.");  
var response = await agent.RunAsync(  
    "Describe a funny story of 3 lines about a dog barking at a cat");  
Console.WriteLine(response.Text);
```


Transforming to Agent Framework

```
#pragma warning disable SKEXP0110  
var afAgent = kernelAgent.AsAIAgent();  
var afResponse = await afAgent.RunAsync("Describe a funny cat in 2 lines.");  
Console.WriteLine(afResponse.Text);
```


Microsoft Agent Framework

Support for (or in preview)

*Open Standards interoperability
Consuming and hosting
MCP, A2A, OpenAI, ...*

Retrieval Augmented Generation

YAML Declared Agents and Skills

Memory

...

Becoming the basis for Agentic Architecture in .NET



The history of Microsoft Agent Framework

May 2025

June 2025

October 2025

February 2026

April 2026

Build 2025

Agentic DevOps vision

Separate systems announced



The history of Microsoft Agent Framework

May 2025

June 2025

October 2025

February 2026

April 2026

Deep research tool was released



The history of Microsoft Agent Framework

May 2025

June 2025

October 2025

February 2026

April 2026

Microsoft Agent Framework announced

Autogen and Semantic kernel in maintenance mode

Ignite 2025

Expanded agent capabilities



The history of Microsoft Agent Framework

May 2025

June 2025

October 2025

February 2026

April 2026

*Microsoft Agent Framework RC announced
New features were added*



The history of Microsoft Agent Framework

May 2025

June 2025

October 2025

February 2026

April 2026

*Microsoft Agent Framework released
Process Framework was announced*



Our story



The September call to adventure



*Request for an **automated evaluation tool**
Providing grounded feedback on YAML pipelines*

*Agreed to use **Microsoft Agent Framework**
Customer started internal team as well*

The January Threshold

*To move faster, we leaned on **GitHub Copilot**.*

*But with **preview technology**, **thin docs**, **shifting APIs**, and outdated examples, Copilot could **not reliably** guide us.*

*It helped with syntax.
It struggled with the unknown.*



The January Threshold

We tried to build the full castle from day one...



The February mentorship



Back to the beginning

*Build the **smallest** useful **version** first*

***Validate** the approach*

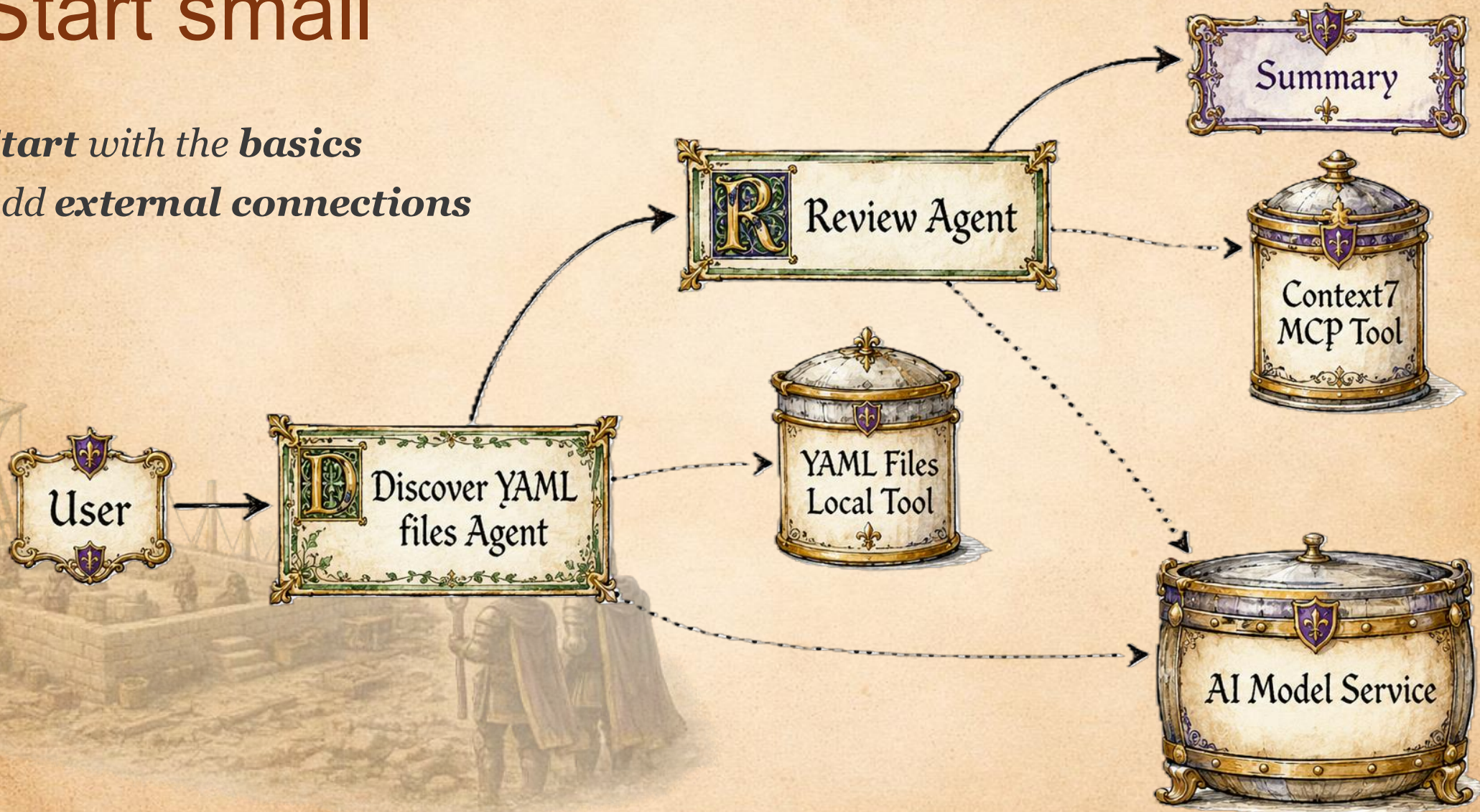
*Then **expand** with confidence.*

The February mentorship

Start small

Start with the basics

Add external connections



The February mentorship

Start small

```
[Description("Get the contents of a YAML file to investigate.")]
static string GetYamlContents([Description("The name of the YAML file.")] string fileName)
{
    return File.ReadAllText(fileName);
}

var fileReaderInstructions = """ """;
var fileReader = chatClient
    .AsAIAgent(
        name: "YAML File Reader",
        description: "An agent that reads YAML files and answers questions about their contents.",
        instructions: fileReaderInstructions,
        tools: [AIFunctionFactory.Create(GetYamlContents)]);
```


The February mentorship

Start small

```
await using McpClient context7McpClient = await GetContext7McpClientAsync(configuration);  
IList<McpClientTool> context7McpTools = await context7McpClient.ListToolsAsync();  
  
var qualityValidationAgentInstructions = """" """";  
var qualityValidationAgent = chatClient  
    .AsAIAgent(  
        name: "YAML Quality Validator",  
        description: "An agent which validates the quality of the YAML file contents.",  
        instructions: qualityValidationAgentInstructions,  
        tools: [.. context7McpTools]);  
  
var workflow = AgentWorkflowBuilder.BuildSequential(fileReader, qualityValidationAgent);  
  
// More information on how to run workflows on Microsoft Learn...
```


The February mentorship Expand with Confidence

Add **structure** to the **output** through **instructions**

Enable Agents to run **concurrently**



The February mentorship Expand with Confidence

// Executors are Implementations to handle messages within the workflow pipeline

// Agents can be converted to Executors

```
var workflow = new WorkflowBuilder(startExecutor)
    .AddFanOutEdge(startExecutor, [agent1, agent2])
    .AddFanInBarrierEdge([agent1, agent2], aggregationExecutor)
    .WithOutputFrom(aggregationExecutor)
    .Build();
```


The March Transformation

A new hero joins the party
Tasked to review, improve and extend



The March Transformation Consolidation

A common project already existed

Used by all samples

But held only a couple of visual tools

Basics of Agent creation kept being duplicated

Configuring of

ChatClient

MCP tool

YAML File Retrieval

Agent Definitions and logic

Perfect to consolidate and reuse



The March Transformation Consolidation

```
public static class Tools
{
    [Description("Get the contents of a YAML file to investigate.")]
    public static string GetYamlContents([Description("The name of the YAML file.")] string fileName)
    {
        ...
    }
}
```


The March Transformation Consolidation

```
public static class ConfigurationRootExtensions
{
    public static async Task<McpClient> CreateContext7McpClientAsync(
        this IConfigurationRoot configuration)
    {
        ...
    }

    public static Task<IChatClient> CreateAzureOpenAiChatClientAsync(
        this IConfigurationRoot configuration)
    {
        ...
    }
}
```


The March Transformation Consolidation

```
public interface IAgentDefinition<out T>
{
    string AgentName { get; }

    string AgentDescription { get; }

    ChatOptions GetAgentChatOptionsWithProvidedTools(IList<AITool> tools);

    ChatResponseFormatJson GetAgentChatResponseFormatJson();

    T? GetOutputFromChatMessage(ChatMessage chatMessage);
}
```


The March Transformation Consolidation

```
<PackageReference Include="Azure.AI.OpenAI" Version="2.1.0" />  
<!-- Keeping the current version to minify consolidation refactoring -->  
<PackageReference Include="Microsoft.Agents.AI.Workflows" Version="1.0.0-preview.260128.1" />  
<PackageReference Include="Microsoft.Extensions.AI.OpenAI" Version="10.2.0-preview.1.26063.2" />  
<PackageReference Include="ModelContextProtocol.Core" Version="0.7.0-preview.1" />
```


The March Transformation Structured Outputs

Structured outputs requested through instructions
Expected JSON being read as generic document

.NET Class could be configured instead
Ensuring

Output Structure

More Consistent Results

Additional Domain Logic



The March Transformation Structured Outputs

A simple Agent call:

```
AgentResponse<Story> response = await storyAgent.RunAsync<Story>(
    "Describe a funny cat in 2 lines");
```

When your Agent is part of a workflow:

```
var chatOptions = new ChatOptions()
{
    Instructions = AgentInstructions,
    ResponseFormat = ChatResponseFormat.ForJsonSchema<Story>(
        schemaName: "StoryResult",
        schemaDescription: "The generated story along with a synopsis of characters"),
    AllowMultipleToolCalls = true,
    Tools = tools
};

// Add ChatOptions to AI Agent creation
// Deserialize from JSON when retrieving responses from agent from results
```


The March Transformation

Built-in Orchestration Patterns

Workflows constructed by custom edges and executors

Using AddFanOutEdge and AddFanInEdge

Too advanced for requirement

Agent Framework provides orchestration patterns

Less maintenance, better optimization

Built-in concurrent workflow sufficient



The March Transformation

Built-in Orchestration Patterns

Preparing custom fan-out to fan-in Orchestration:

// Executors are Implementations to handle messages within the workflow pipeline

// Agents can be converted to Executors

```
var workflow = new WorkflowBuilder(startExecutor)
    .AddFanOutEdge(startExecutor, [agent1, agent2])
    .AddFanInBarrierEdge([agent1, agent2], aggregationExecutor)
    .WithoutOutputFrom(aggregationExecutor)
    .Build();
```

Using Built-In Concurrent Orchestration:

```
var workflow = AgentWorkflowBuilder.BuildConcurrent([agent1, agent2]);
```


The March Ordeal

The time came to extend the samples

We are going to add the web host

Hosting Feature Documentation was scarce

Promised results were worth the challenge

But a sleeping dragon was stirred from its sleep...



The March Ordeal

Updating to include hosting

Newest version at the time:

```
dotnet add package Microsoft.Agents.AI.Hosting --version 1.0.0-preview.260225.1
```

Minimal dependencies:

net10.0

Microsoft.Agents.AI ($\geq 1.0.0\text{-rc2}$)

Microsoft.Agents.AI.Abstractions ($\geq 1.0.0\text{-rc2}$)

Microsoft.Agents.AI.Workflows ($\geq 1.0.0\text{-rc2}$)

Microsoft.Extensions.AI ($\geq 10.3.0$)



The March Ordeal

Updating to include hosting

This meant updating the common project dependencies

```
<PackageReference Include="Microsoft.Agents.AI.Workflows" Version="1.0.0-rc3" />  
<PackageReference Include="Microsoft.Extensions.AI.OpenAI" Version="10.3.0" />  
<PackageReference Include="ModelContextProtocol.Core" Version="1.1.0" />
```

*Introducing **build breaking** errors like:*

```
// Execute the workflow and stream the events as they happen. Before RC
```

```
var run = await InProcessExecution.StreamAsync(workflow, inputMessage);
```

```
// Execute the workflow and stream the events as they happen.
```

```
var run = await InProcessExecution.RunStreamingAsync(workflow, inputMessage);
```

*And new **runtime** unexpected workflow behavior*

Breaking old samples beyond redemption

But Built-In Orchestration Patterns survived

The March Ordeal

Time to create a .NET Webhost

Adding

OpenTelemetry Providers



```
// Agent Framework metrics
builder.Services.ConfigureOpenTelemetryMeterProvider(
    (_, providerBuilder) => {
        providerBuilder.AddMeter("*Microsoft.Agents.AI");
    });

// ChatClient and Agent Framework traces
builder.Services.ConfigureOpenTelemetryTracerProvider(
    (_, providerBuilder) => {
        providerBuilder.AddSource("*Microsoft.Extensions.AI");
        providerBuilder.AddSource("*Microsoft.Extensions.Agents*");
    });

// Add the default OpenTelemetry configuration from Aspire Service Defaults
builder.AddServiceDefaults();

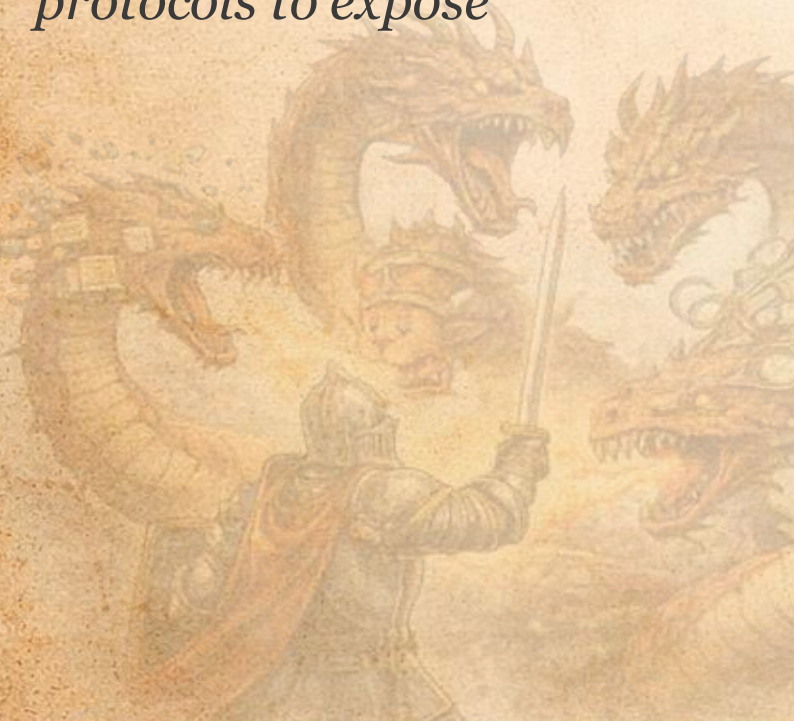
// Additional logging on httpclient calls
builder.Services.AddHttpClient().AddLogging();
```


The March Ordeal

Time to create a .NET Webhost

Time to create a .NET Webhost

*Instrument the different
protocols to expose*



```
// Add OpenAPI
builder.Services.AddOpenApi();

// Add AG-UI (web-based AI agent applications)
builder.Services.AddAGUI();

// Add OpenAI (agents via Chat Completions or Responses APIs)
builder.AddOpenAIResponses();
builder.AddOpenAIConversations();

// Add MCP (Based on classes with attribute in project)
builder.Services
    .AddMcpServer()
    .WithHttpTransport()
    .WithToolsFromAssembly();
```


The March Ordeal

Time to create a .NET Webhost

Time to create a .NET Webhost

Adding reusable ChatClient

Preparing External MCP Tool



```
// Create IChatClient by configuration. Add OpenTelemetry
var chatClient =
    (await builder.Configuration.CreateAzureOpenAiChatClientAsync())
    .AsBuilder()
    .UseOpenTelemetry(configure: c => c.EnableSensitiveData = true)
    .Build();

// Clients are thread safe, register it in DI for general reuse
builder.Services.AddChatClient(chatClient);

// Context 7 MCP tool will be used by multiple agent definitions
await using var context7McpClient =
    await builder.Configuration.CreateContext7McpClientAsync();
var context7McpTools = await context7McpClient.ListToolsAsync();
IList<AITool> tools = [.. context7McpTools];
```


The March Ordeal

Time to create a .NET Webhost

Time to create a .NET Webhost

Register AI Agent Singleton

With

AITools

Structured Output

OpenTelemetry

```
builder.AddAIAgent(YamlBestPracticesCheckingAgentDefinition.AgentName,
    {serviceProvider, _} =>
    {
        var client = serviceProvider.GetService<IChatClient>()
            ?? throw new InvalidOperationException();

        var chatOptions = YamlBestPracticesCheckingAgentDefinition
            .GetAgentChatOptionsWithProvidedTools(tools);

        // Use this to support combo AITools + Structured Output
        var agentOptions = new ChatClientAgentOptions
        {
            Name = YamlBestPracticesCheckingAgentDefinition.AgentName,
            Description = YamlBestPracticesCheckingAgentDefinition.AgentDescription,
            ChatOptions = chatOptions
        };

        var agent = client.AsAIAgent(agentOptions)
            .AsBuilder()
            // Enable OpenTelemetry for this agent
            .UseOpenTelemetry(configure: agent => agent.EnableSensitiveData = true)
            .Build();

        return agent;
    });
```


The March Ordeal

Time to create a .NET Webhost

Time to create a .NET Webhost

Create a workflow

Expose it as an Agent

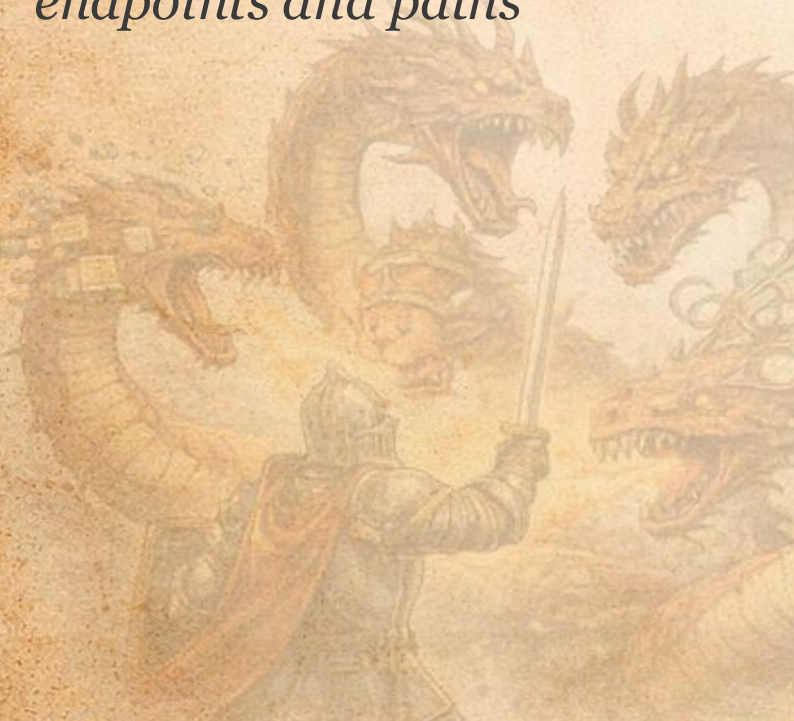
```
builder.AddWorkflow(  
    "validate-yaml",  
    (serviceProvider, key) =>  
    {  
        var agent1Name = YamlVersionCheckingAgentDefinition.AgentName;  
        var agent2Name = YamlBestPracticesCheckingAgentDefinition.AgentName;  
  
        var agent1 =  
            serviceProvider.GetKeyedService<IAgent>(agent1Name) ??  
            throw new InvalidOperationException($"Agent '{agent1Name}' not found.");  
        var agent2 =  
            serviceProvider.GetKeyedService<IAgent>(agent2Name) ??  
            throw new InvalidOperationException($"Agent '{agent2Name}' not found.");  
  
        var workflow = AgentWorkflowBuilder.BuildConcurrent(key, [agent1, agent2]);  
  
        return workflow;  
    })  
    // Workflow needs to be added as agent as well for endpoints  
    .AddAsIAgent();
```


The March Ordeal

Time to create a .NET Webhost

Time to create a .NET Webhost

Map the different protocol endpoints and paths



```
var app = builder.Build();

app.MapDefaultEndpoints();

// Discover the agents from DI for OpenAI
app.MapOpenAIResponses();
app.MapOpenAIConversations();

// Define the endpoints per agent for AG-UI
app.MapAGUI(
    "/validate-yaml/" + agent1Name,
    app.Services.GetKeyedService<AIAgent>(agent1Name) ??
    throw new InvalidOperationException());
app.MapAGUI("/validate-yaml/" + agent2Name,
    app.Services.GetKeyedService<AIAgent>(agent2Name) ??
    throw new InvalidOperationException());

// Register base path for MCP endpoints.
app.MapMcp(pattern: "/mcp");

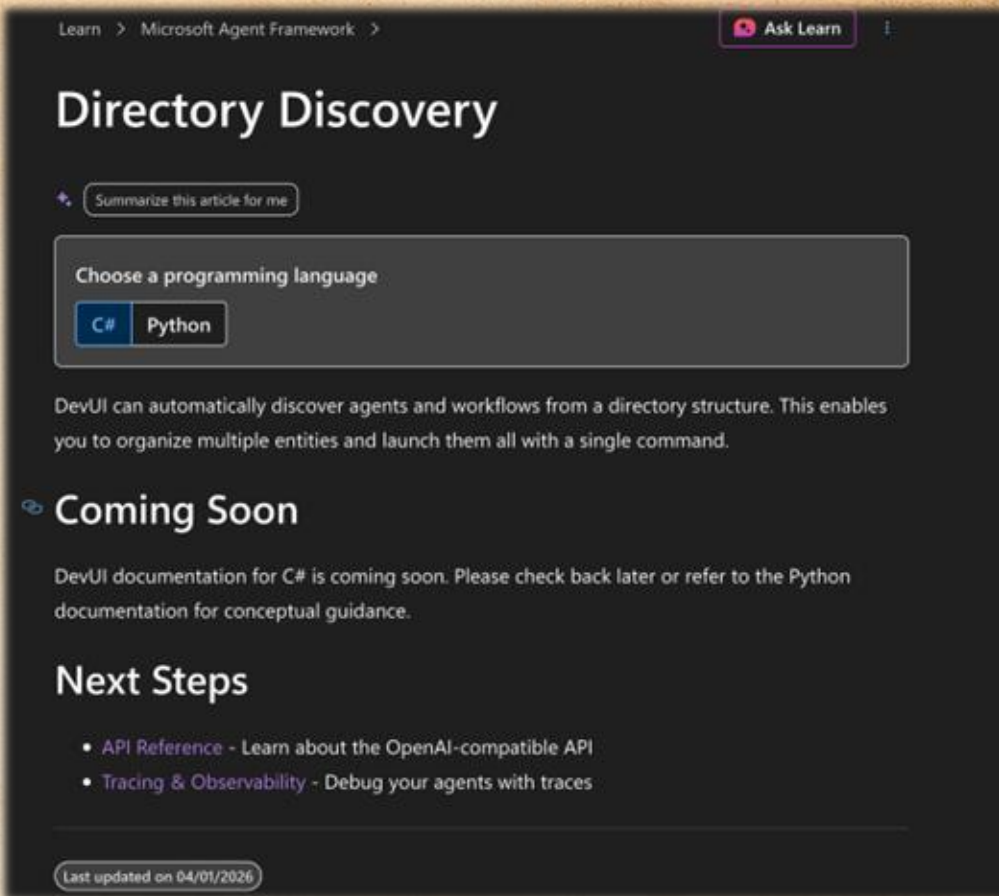
// Register test tooling for OpenAI supported agents
app.MapDevUI();
```


The March Ordeal Adding test tooling

*DevUI is an additional feature to include
Either in Agent webhost application or Aspire
Visualize and test agent and workflow endpoints*

*Official Learn Documentation is lacking
But information to be found through
Videos mentioned on GitHub
Small samples on GitHub*

```
<PackageReference Include="Microsoft.Agents.AI.DevUI"  
    Version="1.0.0-preview.260311.1" />
```



```
app.MapDevUI();
```




Ready to run workflow

No events yet. Start a conversation to see real-time events.

Chat with yami-best-practices-checker

An agent which validates a YAML pipeline against industry best practices.

No conversations



+ New Conversation

Events

Traces

Tools

Events 0

Start a conversation with yami-best-practices-checker
Type a message below to begin

No events yet. Start a conversation to see real-time events.

Message yami-best-practices-checker... (Shift+Enter for new line)



Deployment Guide

The March Ordeal

Adding test tooling

Aspire was used to host the sample

Providing a safe environment to connect and test

Perfect platform to host test tooling

MCP Inspector

```
<PackageReference Include="CommunityToolkit.Aspire.Hosting.McpInspector"  
    Version="13.1.2-beta.518" />
```

```
builder  
    .AddMcpInspector("mcp-inspector")  
    .WithMcpServer(api)  
    .WaitFor(api);
```

But it didn't work out of the box...



The March Ordeal

Adding test tooling

YamlValidation

Update now Aspire 13.2.4 is available. [Upgrade instructions](#)

Ignore X

Console logs

mcp-inspector: Watching logs...

12

13 New StreamableHttp connection request

14 Query parameters: {"url":"https://localhost:7191/mcp","transportType":"streamable-http"}

15 Created StreamableHttp client transport

16 Client <=> Proxy sessionId: 372b4719-8c5a-48cf-bca1-5e9e92c7cd8c

17 stderr Proxy <=> Server sessionId: 0V7x0asFe9_evSvn0G6UVw

18 Received POST message for sessionId 372b4719-8c5a-48cf-bca1-5e9e92c7cd8c

19 Received GET message for sessionId 372b4719-8c5a-48cf-bca1-5e9e92c7cd8c

20 stderr node:internal/webstreams/readablestream:1068

21 stderr throw new ERR_INVALID_STATE.TypeError('Controller is already closed');

22 stderr ^

23 stderr

24 stderr TypeError [ERR_INVALID_STATE]: Invalid state: Controller is already closed

25 stderr at ReadableStreamDefaultController.close (node:internal/webstreams/readablestream:1068:13)

26 stderr at PassThrough.<anonymous> (file:///Users/wouterbauchart/.npm/_npx/7c622f3bacffc9df/node_modules/@modelcontextprotocol/inspector/server/build/index.js:196:28)

27 stderr at PassThrough.emit (node:events:508:20)

28 stderr at endReadableNT (node:internal/streams/readable:1729:12)

29 stderr at process.processTicksAndRejections (node:internal/process/task_queues:90:21) {

30 stderr code: 'ERR_INVALID_STATE'

31 stderr }

32 stderr

33 stderr Node.js v25.6.1

MCP Inspector v0.17.2

Transport Type

Streamable HTTP

URL

https://localhost:7191/mcp

Connection Type

Via Proxy

Server Entry Servers File

> Authentication

> Configuration

> Connect

Connection Error - Check if your MCP server is running and proxy token is correct

The March Ordeal

Adding test tooling

Solution discovered through SKILL.md from online repository...

<https://lobehub.com/skills/jerrynixon-github-copilot-sql-dab-azure-mcp-aspire-mcp-inspector>

*Aspire Extension did not use
latest version of MCP Inspector*



Package

XML

```
<PackageReference Include="CommunityToolkit.Aspire.Hosting.McpInspector" Version="13.1.1" />
```

Installation Method

View Details >

☒ I'm an Agent ☐ I'm a Human

Send this prompt to your agent to install the skill

Agent prompt

Curl `https://lobehub.com/skills/jerrynixon-github-copilot-sql-dab-azure-mcp-aspire-mcp-inspector/skill.md`, then follow the instructions to set up LobeHub Skills Marketplace and install the skill. Once installed, read the SKILL.md file in the installed directory and follow its instructions to complete the task.

[Copy Prompt](#)

Canonical Program.cs Pattern

C#

```
var mcpInspector = builder
    .AddMcpInspector("mcp-inspector", options =>
    {
        options.InspectorVersion = "0.28.0"; // always pin - default version has known StreamableHTTP bugs
    })
    .WithMcpServer(dabServer, transportType: McpTransportType.StreamableHttp)
    .WithParentRelationship(dabServer) // groups under the MCP server in the dashboard
    .WithEnvironment("DANGEROUSLY_OMIT_AUTH", "true") // removes token prompt for local dev
    .WaitFor(dabServer);
```

Table of contents

- Documentation references
- Package
- Canonical Program.cs Pattern
- Transport Type

MCP Inspector v0.20.0

Transport Type

Streamable HTTP

URL

https://localhost:7191/mcp

Connection Type

Via Proxy

Server Entry

Servers File

> Authentication

> Configuration

▶ Connect

● Disconnected

Connect to an MCP server to start inspecting

Need to configure authentication?

Open Auth Settings

History

Clear

2. logging/setLevel

▶

1. initialize

▶

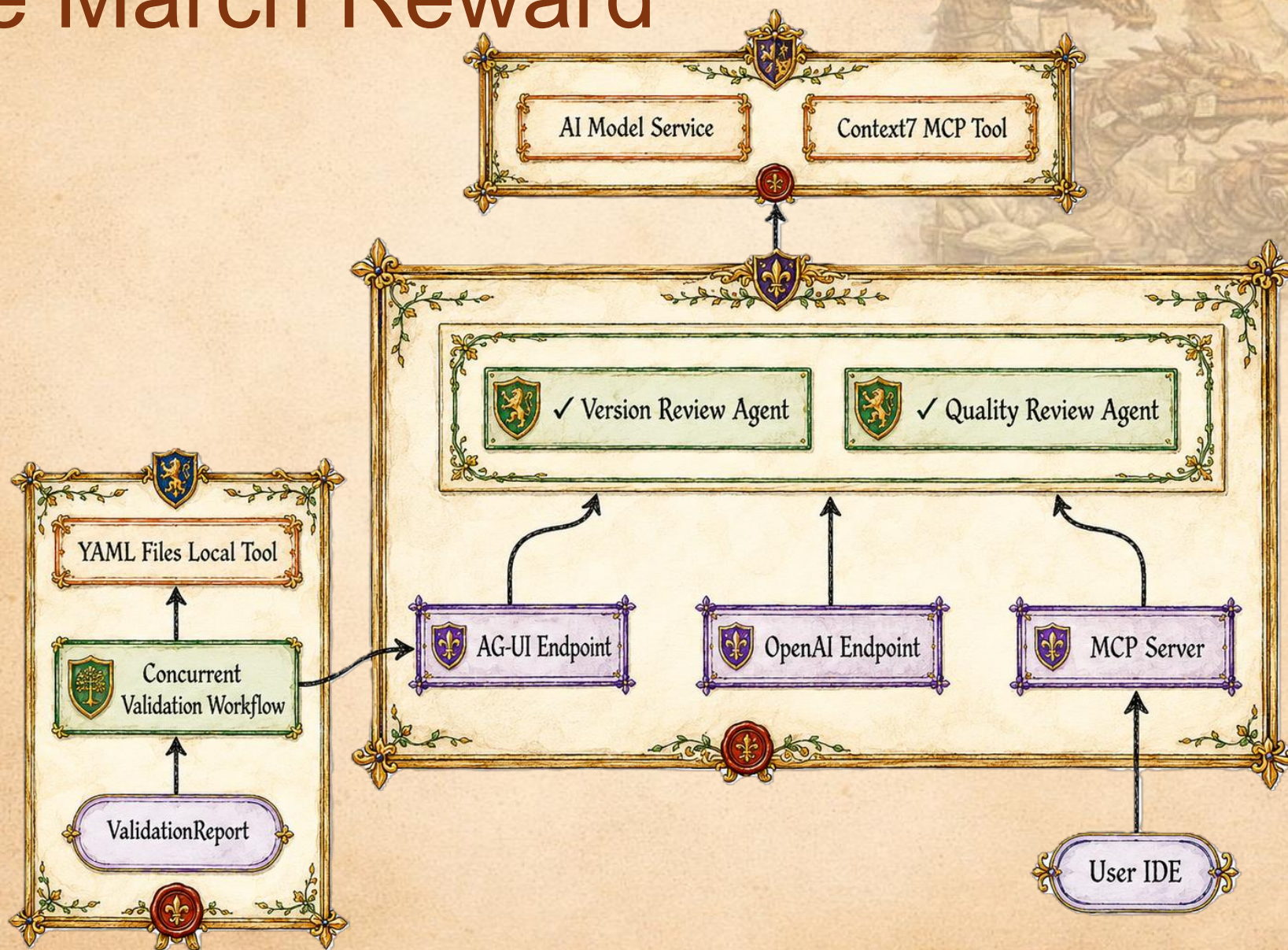
Server Notifications

Clear

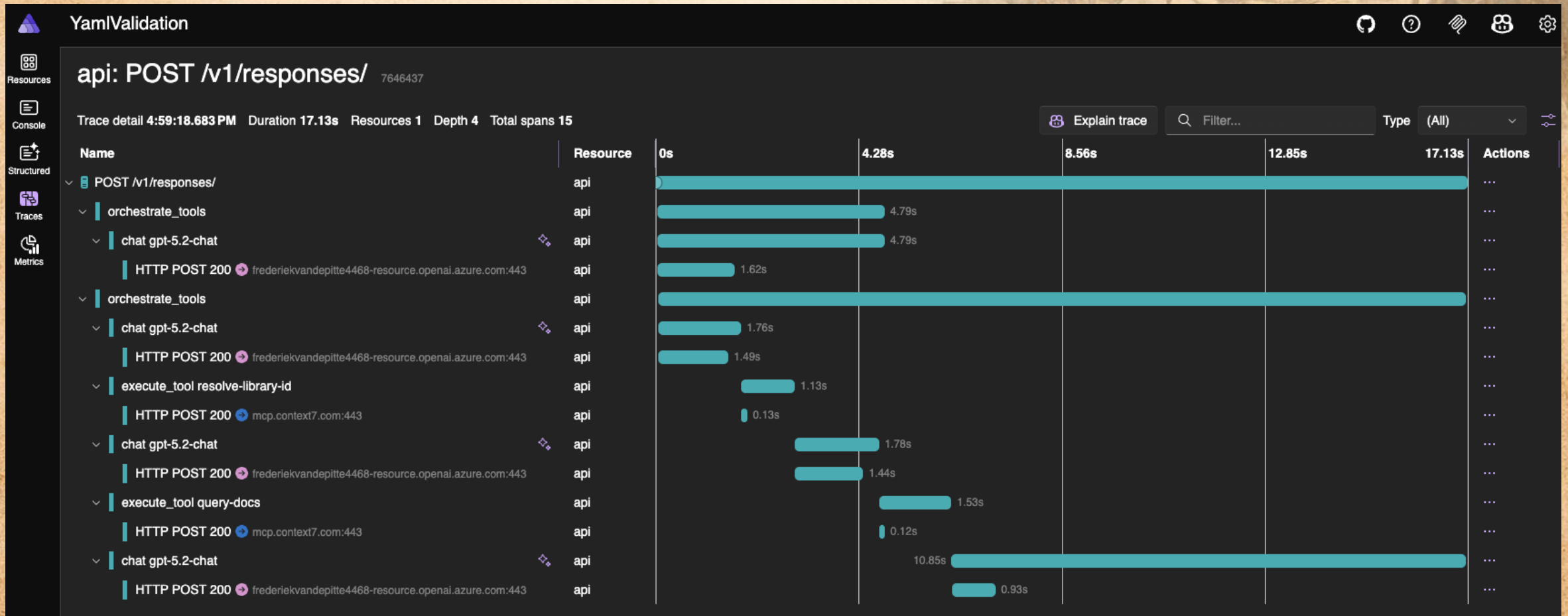
No notifications yet

Light

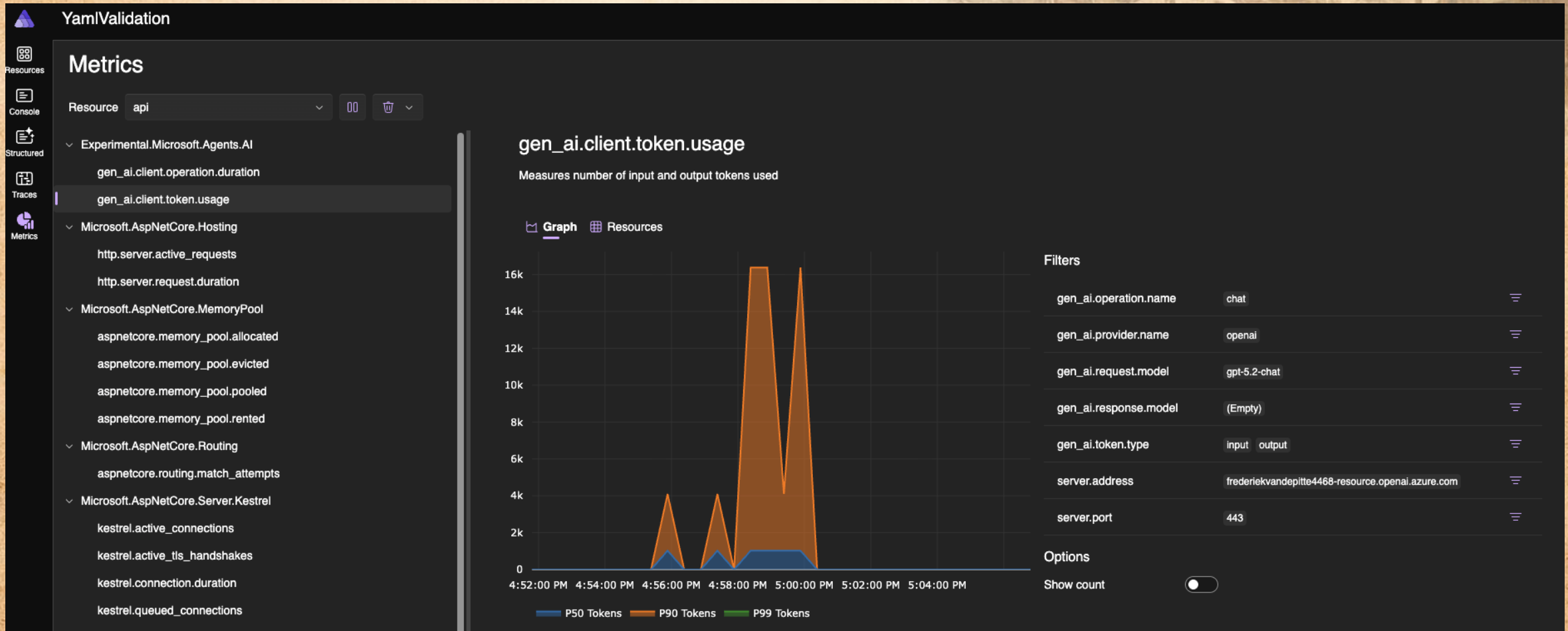
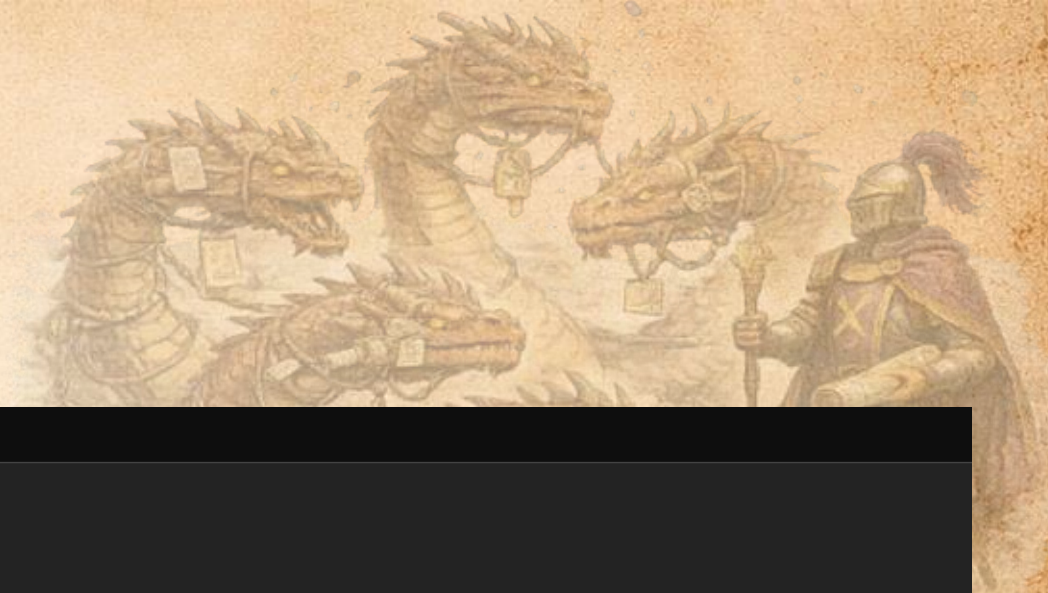
The March Reward



The March Reward OpenTelemetry



The March Reward OpenTelemetry



The April Resolution

Provided demo to customer

*Shared **same experiences** as their development team*

*Proposal for **cooperation** accepted*



The Story Continues...

Together we take the next steps:

Further finetuning of Agent Configurations

Model choices

Instructions & Capabilities

Hosting & Integration

Including solution into dev lifecycle

Monitoring and tracking of usage



Lessons learned



Keep a continuous eye for updates

Follow closely

All Communication Channels

Expect breaking changes

Look beyond official documentation

Embrace Chaos

Start small with Agentic Systems

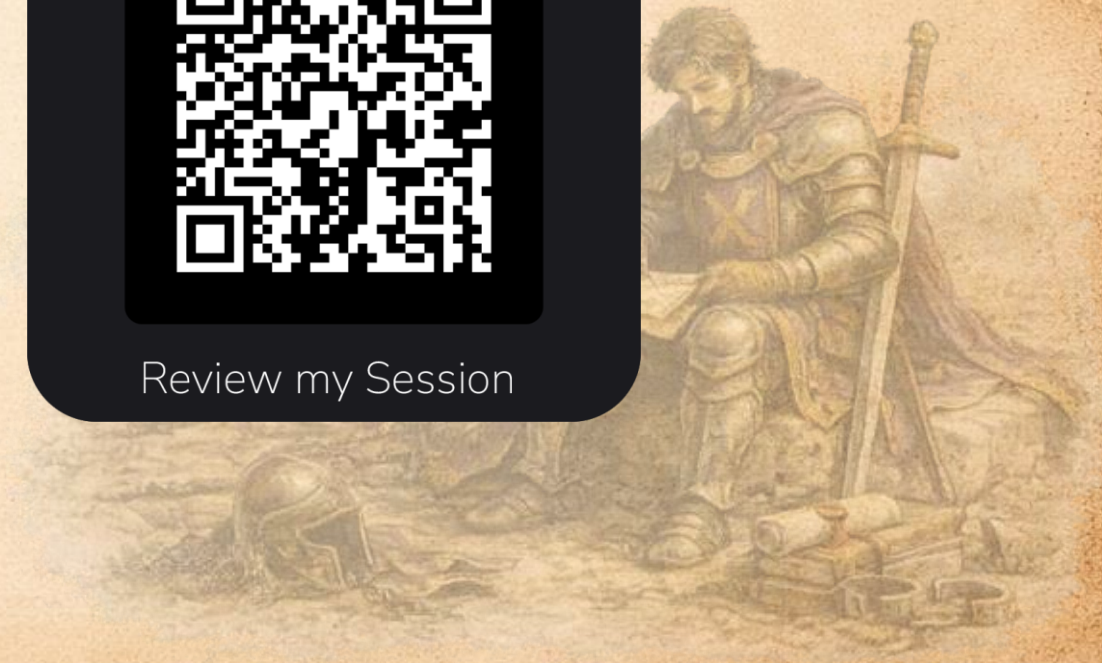
Create Confidence

Learn the differences

Build up Expertise

Enquiries?

Link to slides



Thank you



We hope to see you at our booth

Join us on a next adventure

